

为什么 32 NPU 的 Baseline 看起来很好？

原结果审计、容量可行反例与同输入策略对照

2026-09-07

Contents

回答最初的问题	1
原来的三组实验并不是只差 NPU 数量	2
高利用率具体掩盖了什么	2
每张卡输入多种不同短流之后	3
六种配额与两个种子的完整配对	5
换到 32 卡、六盘之后	5
三个短画像严格同属 SS 的复核	6
其他策略在混合输入上的结果	6
固定画像的机制控制：一个能解释 52% 的输入	6
哪些输入特征值得专门压测	7
其余策略并不保证都好：重分卡可制造新的坏阶段	8
原始 data 也能触发重分卡的严重热点	8
怎样区分可以调度挽回和物理上来不及	9
全输入平均负载低，也可能短时间内全部停算	10
测试方法与可复现性	10
后续怎样评价策略才不容易被平均值误导	10
外部研究与范围	11
复查入口	11

回答最初的问题

32 NPU 的高平均利用率，并不能说明短请求得到良好服务。按每卡多种不同短流重新测试后，画像参数全部来自 data、频率与顺序人工构造的混合输入中，Baseline 整机约 96%，但最短一类请求只有约 58% 的执行时间在计算，约一半不能满足接纳后 SLO。

原 coflow 的 6 盘、种子 20260906 中，Baseline 整机利用率是 93.26%，但完整 192 个短请求从接纳到完成合计只有 30.30% 的时间在计算。Once 已把这些短请求的计算占比提高到 95.76%，只是整机平均只提高约 0.48 个百分点。短请求被救了，平均值却不敏感。

每卡固定画像的机制控制确实可把 32 卡 Baseline 降到约 48%-52%，但这种输入不能代表实际混合请求。因此另增每卡至少三种不同短画像和一至两种长画像、独立打乱完整请求序列的测试。含 1K 外推短画像的六盘混合组中，Baseline 两窗只有 74.53%/80.11%，New once 提高到 91.89%/97.61%；画像参数完全取自原 data 的混合组则整体较高，需要继续看逐类短请求，不能把外推构造的整机数值推广过去。

其他策略能否做好，取决于造成低利用率的原因，也取决于具体策略。能让短读取及时完成、而把仍有计算余量的大读取稍后服务的策略，可以显著改善此类 FIFO 损失；物理容量不足、短时间内所有层都必须读取过多数据、接收链路瓶颈，则不能只靠换 Path 保证全部消失。也不能预设 S1-S3 比 Once 更好。

原来的三组实验并不是只差 NPU 数量

条件	旧 4 卡坏例	multi-SSU near	coflow 正式实验
NPU / SSU	4 / 1	32 / 6、7	32 / 5、6、7
每卡输入	一个固定画像持续重复	22 个请求配额独立乱序	沿用 6 盘 near 的配额
最短计算 C	0.582 ms, 1K 外推画像	1.178 ms, 原 data 格点	同左
最长计算 C	29.857 ms	68.056 ms	68.056 ms
到达	t=0 已有足够积压	每卡构造理想计算时钟	t=0 积压加后续固定工作率时钟
主窗口	每卡暖机 8 请求再等 500ms 后测 1 秒	绝对 [1000,2000] ms	绝对 [1000,2000] ms

共同条件是每请求八层、batch=1、每盘 40 GiB/s、每卡一条合计 50 GiB/s 的接收链路、只预取下一层及已到达的下一请求 L0。Baseline 在**每一盘**都使用自己的 Path0 FIFO；多盘不是合并成一条队列。

multi 的 near 生成器主动把部分 192K/NQL1024 请求换成计算更长的 NQL2048，直到最热盘名义需求降到 39.96 GiB/s 以下。这一步已经改变了输入难度。未调整的原配比直接放大到 32 卡，需求是 316.183 GiB/s，已有结果中的 Baseline 只有 66.779%（六盘）和 79.083%（七盘）。

六盘 near 每卡五个 192K/NQL2048 请求，七盘 near 每卡三个；coflow 对所有拓扑都固定采用前一种配额。因此七盘两轮数据不能作为同输入重复。六盘两轮到达时钟不同，但其全部 704 个 Baseline microbatch 执行时间线逐项相等：后继都已足够早到达，重定时没有影响实际预取。到达口径延迟仍不同，不能互换。

高利用率具体掩盖了什么

下面按原 coflow 六盘、种子 06 的完整 704 请求重新计算。接纳后计算比定义为该画像所有请求的纯计算总时间，除以其接纳到完成总时间；它不是固定秒整机利用率。

画像	请求数	占纯计算时间	Baseline 接纳后计算比	平均 I/O stall
32K / NQL128	128	0.856%	24.35%	29.28 ms
32K / NQL256	64	0.725%	42.57%	21.56 ms
64K / NQL512	64	2.319%	80.43%	12.43 ms
96K / NQL512	64	3.294%	88.76%	9.19 ms
192K / NQL512	128	12.438%	97.30%	3.80 ms
192K / NQL1024	96	18.577%	98.45%	4.30 ms
192K / NQL2048	160	61.791%	99.39%	3.33 ms

两类短请求占 27.27% 的请求数，却只占 1.581% 的纯计算时间；三个 192K 画像占 92.805%。旧例则有一整张卡永久处理短请求，受害者始终占四分之一的卡数。请求数量占比和占据卡时间的比例，是两个不同的量。

窗口内也能直接验证掩盖：原六盘 06 主窗口中，192K/NQL2048 占全部卡时间的 59.63%，自身约 99.69% 在计算；32K/NQL128 占卡时间 3.85%，自身仅约 19.78% 在计算。高平均数是正确的账目，但不能推出每类请求都好。

策略	原整机 U	同一 192 个短请求计算比	短请求平均 stall	短请求接纳 SLO
Baseline	93.26%	30.30%	26.705 ms	55/192
Once, 5 ms	93.74%	95.76%	0.514 ms	184/192
New once, 5 ms	94.64%	97.04%	0.354 ms	186/192
S1	97.63%	93.75%	0.774 ms	184/192
S2	97.26%	94.66%	0.655 ms	184/192
S3	97.99%	94.52%	0.673 ms	183/192

接纳 SLO 为 $\text{completion} - \text{admission} \leq 1.5 \times$ 本请求八层纯计算时间，不含接纳前排队。这项短请求改善覆盖原实验全部六个拓扑/种子组合；例如五盘 Baseline 短请求计算比仅约 13.5%，Once 约 94%。详见可复算的 notes/existing_audit.md 和配套 CSV。

整机单一窗口也不是稳态证明。原六盘种子 07 的 Baseline 首秒为 84.16%，主窗口为 98.73%，完整批次为 90.23%。新的实验固定主窗口，并额外报告第二个连续窗口与完整批次。

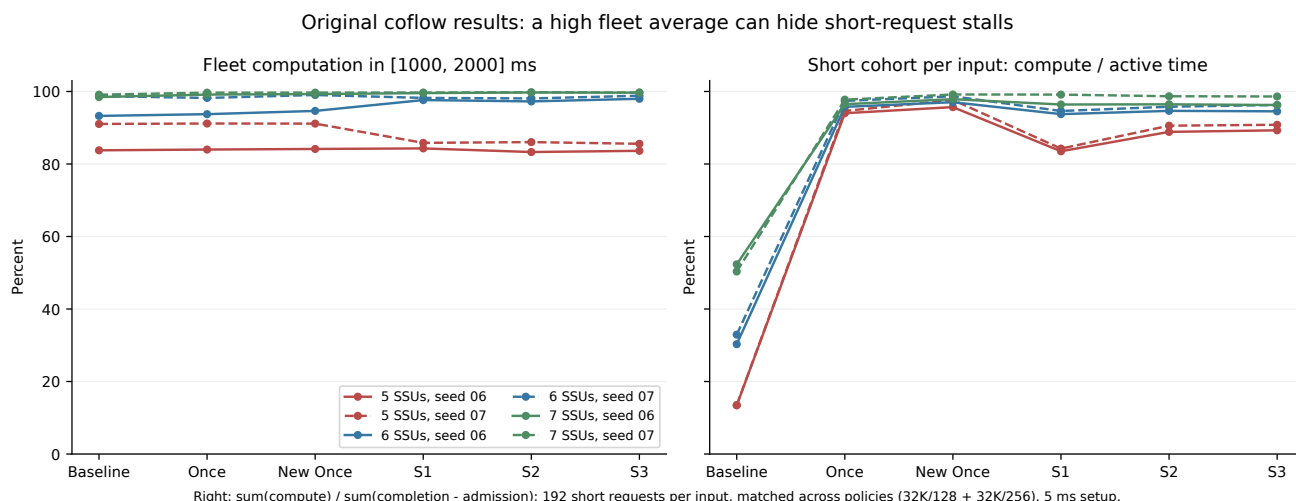


Figure 1: 原实验全部六个拓扑/种子组合。左为固定秒整机利用率，右为同一完整短请求组的接纳后计算占比；两者分母不同。短请求的改善远大于整机平均数所显示的变化。

每张卡输入多种不同短流之后

新增六个配额家族、两个随机种子，每个输入分别运行 Baseline、5ms Once 和 New once。这里测试的是人工配额、人工顺序和 $t=0$ 饱和积压，不是生产到达 trace；“原 data” 仅指画像的 C/V 等参数来自已有表格。每张卡具有相同完整配额，却使用独立的完整请求顺序；先把全部请求放入列表，再用各卡独立随机数打乱一次，没有重复固定的小循环。全部 32 条顺序独立复现且各不相同，所有参数和物理位置跨策略冻结。

外推组 E72/E80 每卡有 1K/NQL128、256、384 三种短画像和 192K/NQL256 长画像。原始组 R33/R38/R58 每卡有 32K/NQL128、256、512 三种短画像，加 192K/NQL1024、2048 两种长画像。原始组 V38 进一步把短画像换为 32K/NQL128、48K/NQL256、64K/NQL512，同时变化计算预算和数据体积。

这里的“较短画像组”是相对 192K 的长计算画像定义的分析分组，不全是模拟器的 SS 类别。代码以序列长度 80K 和 NQL512 为边界，32K/NQL512、64K/NQL512 属于 SL；NQL128/256 的这些短序列画像才属于 SS。表格保留具体画像，不把类别不同造成的取舍说成同一路径类别内部的结论。另设严格三种 SS 原始画像的独立复核，见后文。

家族	短画像来源	短请求占纯计算时间	最热盘需求/容量	C 缩放
E72	1K 外推及 NQL 插值	71.82%	93.54%	无
E80	1K 外推及 NQL 插值	79.86%	70.91%	无
R33	原 data, 三短等请求数	32.92%	89.90%	无
R38	原 data, 三短等请求数	37.74%	96.79%	无
R58	原 data, 短请求数 1:1:8	58.18%	96.13%	无
V38	原 data, 三种短体积	37.55%	97.30%	无

这里“短”按所列画像定义，不等于把全部配额都给最短计算请求。R58 中仅 32K/NQL512 就占 52.54% 的纯计算时间，NQL128 与 256 合计仅 5.63%。V38 中 192K/NQL2048 仍占 53.51% 的纯计算时间。额外统计每个短画像，正是为避免再次掩盖这些差别。



6.436ms 发生在后续七层，占 93.55%；它不是仅有初次加载或请求首层开销。48K/NQL256 和 64K/NQL512 的服务质量明显更好。

甚至逐卡最小利用率也会掩盖这个问题：V38 种子 7 两窗口中最差卡仍有 92.39%/92.65% 在计算，因为每张卡都穿插了长计算请求。若目标只看整机或逐卡平均利用率，这组 Baseline 确实已经较好；不能把短请求的执行比低写成整机 U 低。

V38 种子 7 的三策略完整短请求群体计算比如下，三个单元格都对应同一批 request_id：

短画像	Baseline	Once	New once
32K/NQL128	57.80%	76.04%	92.45%
48K/NQL256	86.93%	94.51%	99.49%
64K/NQL512	98.26%	89.22%	86.53%

前两类改善，第三类却退化。New once 把 32K/NQL128 的 SLO 从 483/896 提高到 870/896，却把 64K/NQL512 从 896/896 降到 808/896；第二个随机种子也复现这一取舍。种子 7 的 64K/NQL512 新增等待约 77%-79% 发生在后续层，只凭这些摘要能定位等待阶段，不能唯一断言是哪条路径的具体排队机制。两个长画像计算比也略降，接纳 SLO 仍为 100%。

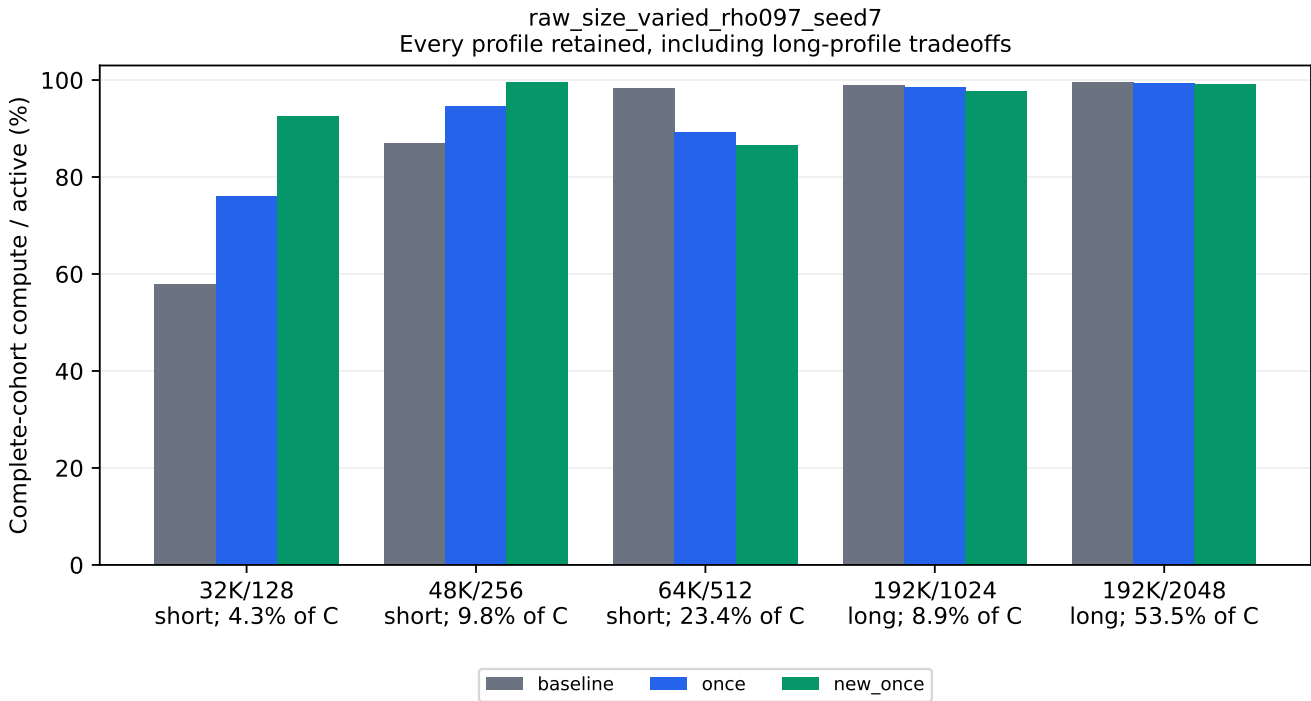


Figure 3: V38 种子 7 保留全部五种画像的收益与代价。纵轴是同一完整请求群体的计算时间/接纳后执行历时，横轴还标出各画像占总纯计算时间的份额；不是整机利用率图。

这组整体收益很小：Baseline、Once、New once 的完整 makespan 分别为 6520.496、6520.297、6489.609ms。Once 只缩短约 0.2ms，New once 约 0.47%。因此“其余策略会好吗”必须结合目标回答：更好地保护最短类，并不保证所有短类都更好，也不保证完整批次有很大收益。

E72 两个种子的 Baseline 共同窗口约 80.29%-82.72%，相较固定流 52% 已经缓解，但仍有持续等待。种子 7 的同输入配对为：Baseline80.29%/82.72%，Once90.68%/92.90%，New once91.62%/93.80%；种子 123 为 81.80%/80.37%、92.46%/94.88%、93.34%/96.65%。全部窗口都全卡 active；后两策略改善约 10-16 个百分点，仍未达到 100%。这是外推参数下的机制扩展，不能标成完全原始 data 结果。

这种改善不是所有画像同时获益。E72 种子 7 的完整群体计算比为：

画像	Baseline	Once	New once
1K/NQL128	69.91%	97.19%	99.99%
1K/NQL256	77.99%	98.59%	99.99%
1K/NQL384	84.78%	99.06%	99.99%
192K/NQL256	93.40%	81.22%	82.30%

New once 三短画像后续层 stall 均为 0、接纳 SLO 均 100%，但长画像平均 stall 从 4.920ms 增加到 14.981ms，SLO 从 576/576 降到 516/576；Once 长画像达标 500/576。另一个种子也复现长画像退化。完整批次仍从 5489.5ms 缩短到 4891.0ms (Once) 和 4785.3ms (New once)，因此这里有可量化的短流保护、长流代价与总体排空收益。

另设仅含三种不同 1K 短画像的独立控制，Baseline、Once 和 New once 两个窗口均 100%，接纳 SLO 也均 100%。这个控制的盘负载仅 14.22%，请求人口也不同；它说明随机多短画像本身并不必然失速，不能据此声称已经在同负载下单独隔离了长流的因果影响。

六种配额与两个种子的完整配对

主实验 36/36 完成，全部 72 个窗口 32 卡全程 active。下表每格依次为固定的第一、第二公共窗口整机 U (%)；按六种预先冻结的配额完整展示，没有删除窗口变差的组合。逐请求和完整批次全表见 notes/mixed_profile_audit/。

输入 / seed	Baseline U1/U2	Once U1/U2	New once U1/U2
E72 / 7	80.29/82.72	90.68/92.90	91.62/93.80
E72 / 123	81.80/80.37	92.46/94.88	93.34/96.65
E80 / 7	86.19/91.96	96.67/97.97	98.43/98.90
E80 / 123	91.53/87.11	96.95/97.07	98.67/98.42
R33 / 7	93.22/97.62	92.86/98.60	93.09/98.83
R33 / 123	97.82/97.42	98.88/98.23	99.05/98.95
R38 / 7	96.35/97.47	97.24/97.95	98.07/98.91
R38 / 123	94.11/94.39	94.29/94.90	95.96/95.08
R58 / 7	96.85/98.38	95.61/96.44	97.06/97.87
R58 / 123	97.81/97.56	95.82/97.74	97.18/97.67
V38 / 7	96.23/96.66	96.36/97.03	97.68/97.42
V38 / 123	95.88/96.58	95.90/97.19	96.58/97.75

例如 R58 种子 7 的 Once 把最短 128/256 两类保护得更好，却使两窗整机 U 从 96.85%/98.38% 降到 95.61%/96.44%，完整批次从 6088.923ms 增至 6193.890ms (慢 1.72%)。占全部纯计算时间 52.54% 的 32K/NQL512 计算比从 98.50% 降到 92.82%，因此“短组整体”的计算比也从 95.51% 降到 93.37%。R38 种子 7 的 Once 则两窗 U 均提高，完整批次仍慢 0.33%。这些是完整配对中的实际取舍，两种种子并不足以推断总体统计显著性。

换到 32 卡、六盘之后

补充两组输入，各运行 Baseline 与 5ms New once。A 沿用八盘 E80 的每一条请求、每卡顺序和所有非 placement 字段，只把拓扑改为六盘并重新条带。B 使用原 data 的三种短体积和两种长画像，重新选择配额以保持最热盘平均需求低于容量。B 因此是新请求人口，不能称为只改变盘数的控制。

表中 A 为 1K 外推多短画像，短 C 占 79.86%；B 为原 data 多体积，短组 C 占 20.49%。U1/U2 仍为两个固定公共窗口，完成时间单位为 ms。

输入	最热盘 ρ	策略	U1	U2	完成时间 (ms)
A	95.20%	Baseline	74.53%	80.11%	5391.77
A	同上	New once	91.89%	97.61%	4473.27
B	96.88%	Baseline	95.72%	96.25%	7778.43
B	同上	New once	98.41%	96.68%	7734.47

全部窗口 32 卡持续 active、idle 为 0。A 的两窗收益均约 17.4 个百分点，完整批次缩短 17.04%；这是每卡多种不同短画像、独立乱序后仍存在明显整机损失的直接配对结果。A 的全部 18,720 个短请求均达到 New once 接纳 SLO，但长画像计算比从 93.26% 降到 77.94%，长请求 SLO 从 384/384 降到 297/384。

B 再次表明整体指标会掩盖请求差异：Baseline 的 32K/NQL128 计算比只有 43.88%、SLO178/576，New once 提高到 88.42%、552/576；48K/NQL256 从 73.53% 提高到 97.78%。但 64K/NQL512 从 94.78% 降到 87.94%、SLO559/576 降到 520/576，两个长类的等待也增加。完整批次只缩短 0.57%，不能把短请求收益说成整机吞吐大幅提升。

A 与八盘 E80 还逐策略核对了同一 Python 环境、输入、核心源码及策略配置。Baseline 从八盘 86.19%/91.96% 降到六盘 74.53%/80.11%；New once 从 98.43%/98.90% 变为 91.89%/97.61%。这里同时改变总容量和块

到盘的映射，只能确认这次拓扑变化的整体影响，不能单独估算二者各贡献多少。这一补充只有一个种子、两个策略；独立复核和全部逐类代价见 [notes/mixed_six_ssu_review.md](#)。

三个短画像严格同属 SS 的复核

使用原 data 的 32K/NQL128、48K/NQL256、64K/NQL128 三种 SS，仍与两个 192K 长画像混合，两个种子各运行三策略，共 6/6 完成。每卡独立打乱 93 条请求，32 卡均有三种短画像，最热盘平均负载 95.19%、无 C 缩放。它改变了第三短画像和整个配额，短组纯计算份额降至 17.70%，192K/NQL2048 占 70.52%；它检验同类别内部是否仍有差异，不能单独归因 V38 的代价就是 SS/SL 分类造成。

真正 SS 画像 / seed	Baseline 同群体计算比	Once	New once
32K/NQL128 / 7	64.17%	69.04%	75.74%
48K/NQL256 / 7	89.12%	90.28%	95.45%
64K/NQL128 / 7	67.83%	78.59%	83.75%
32K/NQL128 / 123	60.62%	64.08%	67.57%
48K/NQL256 / 123	87.28%	87.66%	91.20%
64K/NQL128 / 123	64.33%	73.25%	76.74%

Baseline 整机两种子、两窗口仍有 94.23%–97.73%，最短类的多数 stall 发生在后续层。Once/New once 提高了各 SS 类计算比，但并未消除等待。种子 7 两个长画像的计算比由 98.69%/99.31% 降为 New once 的 93.50%/97.74%，它们的接纳 SLO 仍全过。

同一画像的平均计算比提高，也不保证阈值达标率提高。种子 123 的 48K/NQL256，Baseline 接纳 SLO 为 803/864，Once 为 786/864、New once 为 792/864，虽然表中平均计算比都提高。这是完整相同请求群体的真实结果，不能只用平均 stall 替代延迟分布。两种策略整体 SLO 均提高；完整批次则两个种子都由 Once 更快：Baseline/Once/New once 分别为 7519.96/7434.96/7461.44ms 和 7613.86/7541.35/7585.64ms。全部画像和六格结果见 [notes/mixed_strict_ss_audit/](#)。

其他策略在混合输入上的结果

固定画像的机制控制：一个能解释 52% 的输入

这一节每张卡持续重复自己的固定画像，专用于隔离竞争机制。基于旧例另构造三短一长：不仅改变受害者卡数，也把短 NQL 从 169 改为 128、背景改为 192K/NQL256。每组四卡接到一盘；前三卡重复短画像，第四卡重复长画像。八组即 32 卡、八盘。

卡的角色	每层读取	每层计算 C	理想持续带宽
三张短卡，各自	1.203125 MiB	0.527909 ms	2.225624 GiB/s
一张长卡	263.656250 MiB	8.708366 ms	29.566604 GiB/s

总需求是 $3 \times 2.225624 + 29.566604 = 36.243477$ GiB/s，只占盘容量 90.61%。短画像沿用旧实验的 1K 外推方法，但使用 NQL128，使每个命令恰好 176 KiB；长画像 192K/NQL256 直接来自 data。它是有意构造的机制反例，不声称是生产请求分布。

短层独占 SSD 只需约 0.0294 ms，本来远短于 0.5279 ms 预算。大层却包含 1534 个命令，总 SSD 服务约 6.4369 ms。若许多大层命令已经进入同一 FIFO，后来释放的短层会等待它们；短卡的计算早已结束，必须 stall。

这里不能把 263 MiB 描述成一个不可抢占命令。模拟器的不可抢占单位是一个 176 KiB 块，服务约 4.196 微秒；长等待来自排在前面的许多块和层的末块到齐约束。Once 可把紧急小读从这类积压中隔离出来，同时大读仍有 8.708 ms 计算预算可用。

共同 [1000,2000] ms，4 卡与 32 卡两个局部布局的 Baseline 都为 52.2096%；每张短卡约 36.28%，长卡 100%。共同 [2000,3000] ms 为 52.2160%，不是启动瞬间。八份局部复制没有改变每个子系统，增加 NPU 数本身没有产生保护。

把相同三短一长画像改为跨八盘均匀条带，Baseline 仍为 52.21%；更换为原生 hash，最热盘名义需求 39.1861 GiB/s、本来已低于容量 98%，无需缩放 C，Baseline 约 50.96%。因此不只是“局部绑定一盘”才能构造坏例。32 卡六盘条带输入按最热盘把 C 统一乘 1.24932 后，每盘需求 38.41–39.20 GiB/s，最大单卡接收需求仅 50 GiB/s 容量的 47.33%，Baseline 主窗口仍只有 47.69%。六盘的 C 缩放是明确的构造变体。

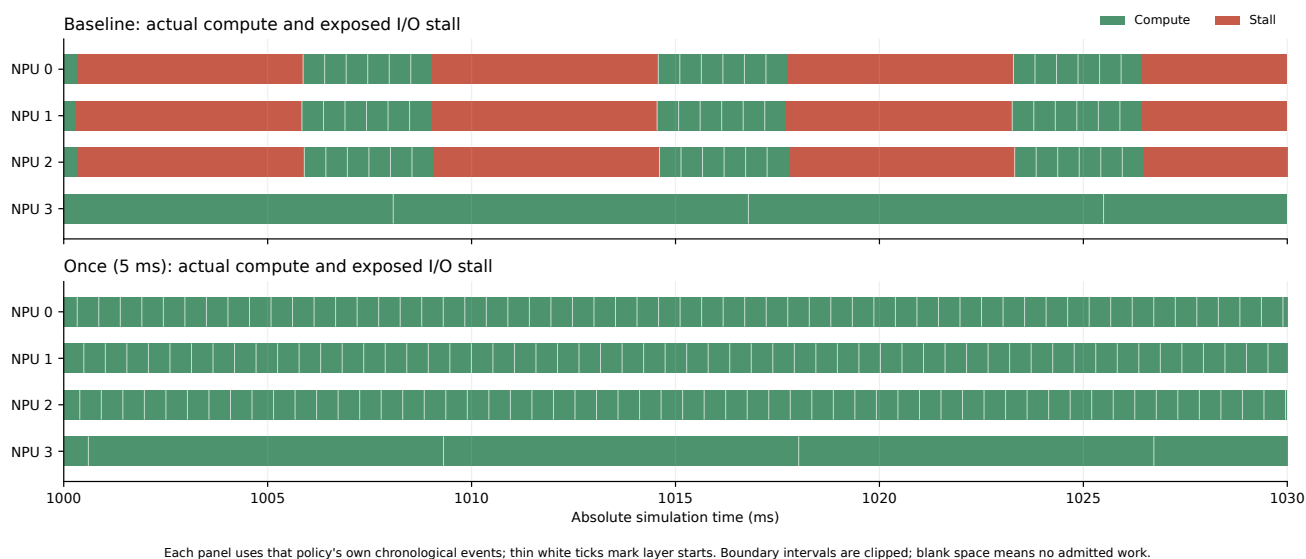


Figure 4: 32 卡八盘输入中同一局部组的四张卡，真实绝对时间 1000-1030ms。绿色为实际计算、红色为暴露 I/O 等待，细白线标记每层计算起点；图边缘截断了跨界区间。两策略使用同一输入，但此时各自处理到的请求编号不同，没有将计算和等待重新打包排列。

相位能缓解，却不必消除问题。八盘条带输入的每张卡启动时间在 0-10ms 独立随机偏移，保持画像、placement、请求人口和提交 seed 不变；两个偏移种子的 Baseline 主窗口都约 75%，第二窗口约 74.98%。相较完全同时启动时 52.21% 明显改善，但 24 张短卡仍约 66.6% 计算、8 张长卡 100%。

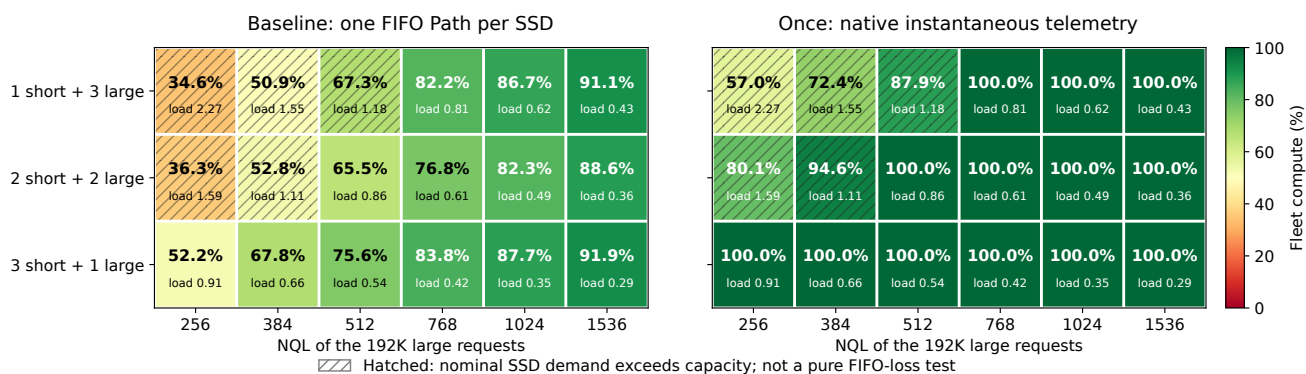
同一随机启动输入上，5ms Once 两个窗口分别为 95.32%/95.00%（偏移 seed7）和 96.32%/95.36%（偏移 seed123），较对应 Baseline 提高约 20-21 个百分点。所有 32 卡在全局八个策略窗口都持续 active。偏移随机种子与提交 seed 分开，后者始终为 42。

哪些输入特征值得专门压测

最容易暴露 FIFO 问题的组合是：**短计算预算、低自身读量的请求持续占据较多 NPU；同一组 SSD 上又有周期性释放大量块的背景请求；这些大请求仍有较长计算余量，读得太早会挤压前者的期限。**可用实际前方队列服务时间与短计算预算的比值筛选，再用仿真验证，不能把一个画像永久标为“坏请求”。

本次还做了 18 个 4 卡输入、两策略共 36 次完整格点测试。固定短画像为 1K/NQL128，改变短卡数与大请求 NQL，13 个输入满足平均盘和链路容量条件；原生 Once 在这 13 组的两个公共窗口均 100%，Baseline 最差 52.21%，另有两短两长输入在 86.21% 名义盘负载下只有 65.55%。

4 NPUs / 1 SSD: victim count and read/compute balance both matter



All cases use [1000,2000] ms, full active coverage, seed 42. 1K short compute is extrapolated; some large NQLs are interpolated.

Figure 5: 完整格点，不删除过载项。格内大数字是全卡计算利用率，小数字是名义盘负载比；斜线格的平均盘需求已超容量。这里 Once 使用即时遥测，与主 32 卡的 5ms 对照分开。

受害者更多与每名受害者受损更重，不是同一件事。例如固定长 NQL768，短卡数 1、2、3 对应名义负载 0.808、

0.613、0.418，Baseline 利用率却依次为 82.21%、76.82%、83.76%。第二项负载更轻、每张短卡也更好，但受害者数量翻倍使整机平均更低；再减少背景大流才使平均回升。因此不能只按 NPU 数、短请求数量或总负载预测结果。

其余策略并不保证都好：重分卡可制造新的坏阶段

旧 4 卡输入补齐尾块，完整输入与物理位置不变，仅更换策略，已经能看到明确反例：

策略	主窗口 U	完整批次完成时间
Baseline	76.23%	8391 ms
Once, 5 ms	99.78%	4589 ms
New once, 5 ms	99.99%	4576 ms
S1, 固定分卡	95.68%	5336 ms
S1, pipeline 重分卡	54.01%	5802 ms
S2, pipeline 重分卡	54.01%	5802 ms
S3, pipeline 重分卡	54.01%	5802 ms
A : deadline 原生控制	100.00%	4582 ms

重分卡后的四张卡在主窗口都持续 active，没有输入耗尽。实际时间线显示，原来一张卡长期处理短请求、一张处理大快请求、两张处理大慢请求；pipeline 将 t=0 先到达处理的短请求分散到四卡，随后四卡在约 1118-1123 ms 几乎一起进入大快画像。主窗口的 4000 NPU-ms 中，3519 NPU-ms 变成 192K/NQL368，只有约 47.72% 在计算；原来提供较长计算余量的 192K/NQL896 在这个窗口完全没有执行。

因此，**均衡完整计算总量，不等于均衡每个时刻的 I/O 需求**。同一个全输入平均资源配比，能被重分卡排成不同的高需求阶段。此例 S1 固定分卡与 pipeline 的受控对照支持这一解释，而不是看到低 U 就归因为尾部空闲。

也必须保留另一半结果：S1-S3 虽然这一秒比 Baseline 更差，完整批次仍比 Baseline 快，但慢于 Once。这里不能简单给出所有指标共用的策略排名。数据和窗口按角色账目保存于 notes/assignment_phase_failure.json。

原始 data 也能触发重分卡的严重热点

使用完全取自 data 的四角色组：两张卡 (32K,NQL512)，两张卡 (192K,NQL1024)；复制为 32 卡、八盘，每组固定数据在一盘。每盘名义需求 37.892 GiB/s，画像未做外推、插值或 C 缩放。固定分卡 Baseline 主窗口约 86.99%，而原 multi-SSU 的 B (deadline 加 fluid 选卡) 在同一冻结输入上仅 10.94%，完整批次也从 5779.8ms 变为 15723.3ms。

该反例的重要触发条件是**全部请求 t=0 到达**。模拟器将同刻 arrival 按原 NPU、request_id 排序；B 先看到原 NPU0 的一整队 SSU0 请求，把最初 32 条分派给 32 张执行卡。由于每卡按自己的接纳队列处理，主窗口全部 32 张卡的当前请求都来自 SSU0；SSU1-7 没有与该窗口重叠的读取生命周期。不是八盘总容量不够，而是在线选卡在这种到达顺序下制造了单盘工作阶段。

独立核对了全部 2736 请求、2651 次重绑，以及逐请求 I/O 完成计数合计 8,292,352 条、逐盘字节总量和提交 placement 断言；这不是逐条物理服务时间线审计。原 SSD placement 未改变；主窗口有 32000 NPU-ms 已接纳工作，其中 3501.05ms 计算、28498.95ms 等待，idle 为 0。不能用全程尾部空闲解释这一秒的低 U。

进一步只把到达按 (generation, 原 NPU) 交错展开，每条相隔 1ns，最后一条到达为 2.735 微秒；C、V、placement、原 NPU、ID、每卡顺序全部保留。两个策略在这一**新输入**上重新配对运行：

到达构造	Baseline U [1,2]s	B U [1,2]s	Baseline U [2,3]s	B U [2,3]s
全 t=0, 原 NPU 分组 ties	86.994%	10.941%	86.739%	12.989%
2.735 微秒内交错展开	86.994%	99.106%	86.739%	94.831%

Baseline 几乎精确不变，而 B 的巨损消失。这支持“同刻到达处理顺序经过选卡和 FCFS 队列放大成 SSU 热点”的机制解释；不能把旧输入的 10.94% 泛化为任意到达下 B 都差，也不能只展示新输入 99.11% 而隐去其脆弱性。仅打乱 manifest 数组不会产生此对照，因为原模拟器会重新按原 NPU 排序。

四格最终统一为本机同一 Python 环境：Baseline 两格均用同一原生 Path0 包装器，B 两格均用同一原生 deadline+fluid 实现。此外，对原 raw 输入补跑本机共享包装器，全部 2736 请求指标、所有 microbatch/层时间线、窗口和 SLO 与本机原生 Baseline 精确相同；远端共享包装器的请求/层时间线也精确相同，仅窗口汇总存在浮点归约末位差异。这是本输入的有限桥接，不声称所有策略跨版本逐位一致。新到达 B 的完整完成时间为 4966.8ms，原/新 Baseline 分别 5779.778333/5779.778361ms。

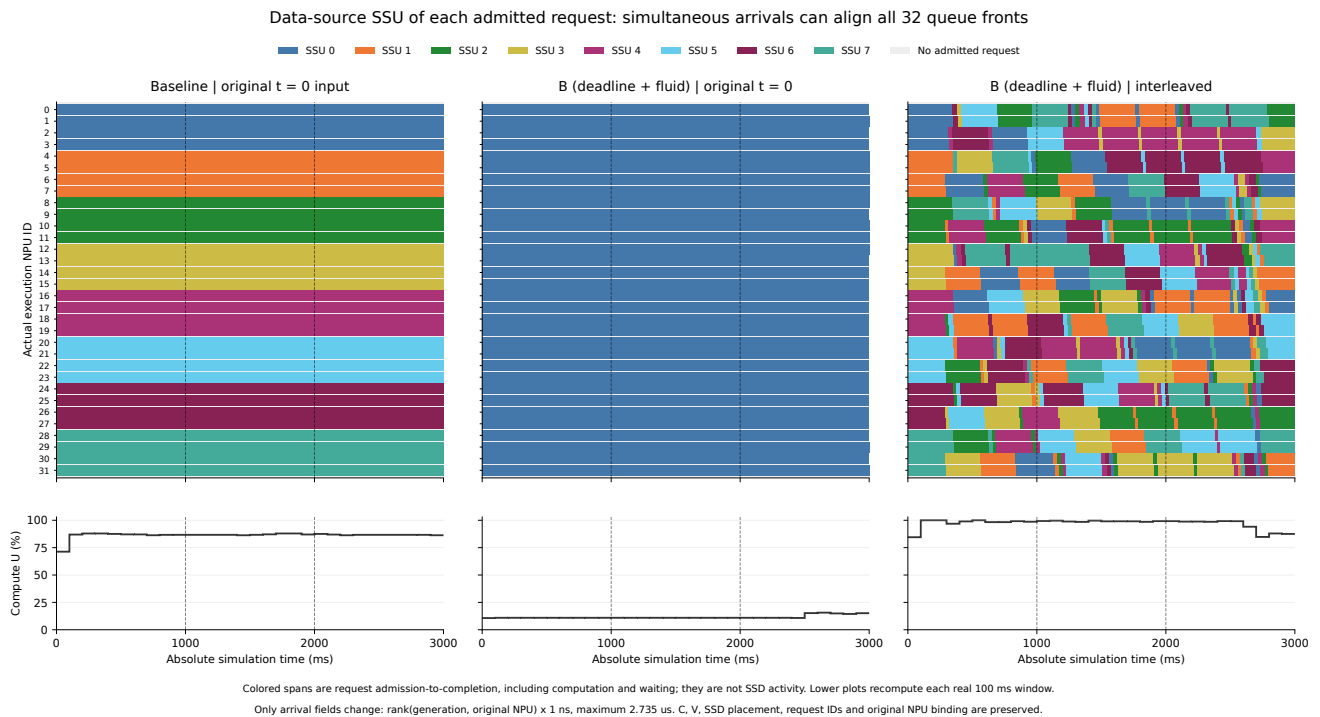


Figure 6: 同一原始 data 人口的三种执行结果。上排按每张真实 NPU 当前已接纳请求的数据来源 SSU 着色，包含该请求的计算和等待，不是 SSD 忙时间；下排为逐 100ms 窗口实际计算占比。原 B 在这三秒把所有卡的队首集中于 SSU0，微小到达展开后的 B 则分散到八盘。

怎样区分可以调度挽回和物理上来不及

对已在算的当前层，下一层的暴露等待满足：

$$\text{stall}_i = \max(0, L_i - C_{\text{current}}),$$

其中 L_i 是从下一层可开始预取到最后一块到 HBM 的实际延迟。若后继请求已在当前层开始计算前到达，并触发跨请求预取， L_0 的预算是前驱最后一层的计算时间，不是新请求自己的长计算时间。本轮 $t=0$ 饱和输入满足这一条件；晚到请求不能自动获得完整的前驱计算预算。

平均资源检查是：

$$\sum_i V_{is}/C_i \leq B_s, \quad \sum_s V_{is}/C_i \leq B_{\text{link}}.$$

前者逐 SSD 检查，后者逐 NPU 接收链路检查。这是给定固定画像持续无 stall 的必要条件，不能保证每次有限 deadline 都满足。实际 L_i 还包含排队、客户端发射、跨盘最后分量与接收链路排队。

另一个必要条件针对时间区间：如果某盘必须在 $[a, b]$ 内完成的层，既不能在 a 前释放、又都要在 b 前到齐，那么这些层需要的字节总和不能超过 $B_s(b - a)$ 。超过时，固定这批释放和 deadline 的任何调度都无法全部满足；改变此前的执行相位或预取深度，则可能改变这个局面。

因此需要分清三类低 U：

1. **FIFO 分配不合适**。盘平均有容量、紧急小读自身也能及时完成，却被有较长计算余量的大读挡住。本文三短一长是主要反例。
2. **瞬态必须完成的工作太多**。许多卡同时从短计算请求切换到长 L0，只有一层 lookahead，即使全输入平均负载低，也可能出现短时间内不可满足的期限。
3. **持续物理过载**。每盘或每卡链路的平均需求超过容量，调度无法制造带宽。热点不能只看所有盘的总量，选卡也不会自动改变数据所在盘。

全部换成“曾被堵住的短画像”尤其容易落入第三类。原 data 的 32K/NQL128 每卡需要 36.333 GiB/s，32 卡共 1162.65 GiB/s，而六盘只有 240 GiB/s。同画像持续流的字节守恒上限约 20.64% 平均计算占比。这个上限适用于同画像持续工作，不能直接套到异构有限一秒窗口。

最新多短流也一样需要先查容量：从 V38 人口中删除两个长画像，只保留 2688 个三种短请求，实际 SSD 总读量为 1377.578125GiB、总纯计算时间为 73350.728 NPU-ms。八盘合计 320GiB/s 使完整 makespan 至少 4304.932ms，因此**完整运行**整机 U 不可能超过 53.2461%。这是直接按原 data 和冻结 placement 算出的物理界，不是新增仿真结果，也不约束任选中间一秒。原始画像的短流具有较高 V/C，不能把它们的时间份额任意调高后仍声称平均容量充足。

以仅有 32K/NQL128 与 192K/NQL2048 两类的理想配比举例，保持 32 卡平均不过载时，前者最多占 19.18%（八盘）或 11.51%（六盘）的纯计算时间；这个算术进一步说明长计算时间为何容易主导平均数。它只是平均容量必要条件，尚未计入逐层期限与实际排队。完整多画像配比的推导见 notes/mixed_capacity_explanation.md。

反过来，小请求自己不一定是“坏输入”。需要看它和哪些请求同时竞争、在何种期限下释放；低带宽小请求单独运行通常没有大读造成的 FIFO 积压。

全输入平均负载低，也可能短时间内全部停算

另一组完全取自 data 的 32 卡八盘循环输入，每卡相同四画像配额，只比较共同循环顺序与每卡独立打乱循环顺序。最热盘名义负载仅 55.75%。Baseline 在 [100,1600]ms 的大窗口分别为 96.49% 和 98.81%，看起来都很好；但同序输入最多 32 卡同时 stall，最差 10ms 平均 U 仅 16.63%，独立打乱最多 11 卡同时 stall，最差 10ms 为 77.98%。

对于同序输入的一波 32 个大请求 L0，实际 release 和 deadline 全部落在 [324.617166, 332.414758]ms 中。每盘必须处理 1.020508 GiB，区间内只有 0.311904 GiB 服务容量，相差 3.27 倍。这是逐请求保留了 release、deadline、物理字节的条件容量证据：冻结这一波相位时任何重排都不能全部满足，改变此前的相位则可能改变这些期限。

原生 Once 在这两个大窗口分别为 96.18% 和 98.33%，略低于 Baseline，同时部分短请求 SLO 改善。它说明平均利用率、局部停顿和阈值达标率会有不同排序。这组只用于机制补充，不替代主实验固定 [1,2] 秒窗口；完整四 case 和容量证据保存于 pilot/order_probe。

测试方法与可复现性

本次测试使用原模拟器和原策略实现，新增输入构造、进程调度、只读重聚合脚本。本机与授权远端并行运行，仿真按进程隔离，冻结 manifest 后跨 Python 版本读取并检查输入指纹。

主测试全部请求在 t=0 已到达，按原 NPU 提供至少四秒纯计算工作，作为饱和和队列机制测试。全部策略使用同一人口、同一 placement、同一 submit seed；没有按策略重新生成工作负载。全量完成后保存请求与层级时间线，窗口内逐卡检查 compute+stall+idle。全量 t=0 积压的 arrival 延迟不代表线上到达体验。

原生 Once 使用即时压力信息；coflow 里的 Once/New once 及 S1-S3 使用 5 ms 周期 collector。S1-S3 正式参数沿用 pipeline 选卡、1 ms 信用窗口、urgent_short；额外 fixed 版本只去掉重分卡，明确标为消融。multi 的 deadline 系列使用原来的更及时控制条件，不能解释为同样 5 ms 权限下的公平算法排名。

旧输入包含不满 176 KiB 的尾块，当前 coflow 适配器不支持这种输入。因此旧 exact 只比较原生支持策略；另生成明确标识的 padded 输入，先用 Baseline/Once 做桥接。四卡补齐后名义负载从 99.7154% 变成 99.9759%，仍在容量内，Baseline 主窗口数值保持到所报精度一致。对齐三短一长本来全是 176 KiB，无此补齐。

按最热盘负载缩放 C 的输入，明确标为容量归一化的构造变体；原 data 参数被缩放后不再称为原始实测格点。同配额顺序对照使用完整 cycle 和固定 striping，避免 request_id 重排连带改变 hash 分盘量。

固定角色、同序循环和独立打乱循环是三种不同输入。只有后两者保持每卡相同完整画像配额，能用于顺序对照；固定角色模式按各自 C 提供同等计算时长，短角色有更多请求。所有结果中的“全输入相同”均指同一构造内部跨策略比较。

后续怎样评价策略才不容易被平均值误导

建议把已有 near 输入与本轮每卡独立乱序的多画像输入纳入常规测试，逐类保留原 data 的多体积短画像与严格 SS 组。固定角色、同配额不同循环顺序、同数据位置但不同到达分组的输入，分别用于隔离 FIFO、相位和选卡机制；额外保留明确标识的持续过载组。不能因为构造固定流更容易得到大幅整机损失，就用它替代用户关心的混合请求。

每组至少同时看四项：固定公共窗口的逐卡 compute/stall/idle；按相同 request id 与画像统计的接纳后计算比和 SLO；完整批次 makespan 与全程计算占比；arrival 到 completion 的延迟及接纳排队。对允许重分卡的策略，物理 NPU 编号相同不意味着处理的是同一请求，不能只比较两张逐卡热图下结论。

找 Baseline 坏输入时，可以先筛选“前方大读的服务时间明显超过短计算预算、而每盘平均需求仍不过载”的组合，再扫描短受害者占卡比例、启动相位与数据布局。测试集和调度参数冻结后保留所有策略结果，不按测试种子重新挑最有利的窗口或参数。本文没有提出一个保证无 stall 的新策略。

外部研究与范围

Varys, SIGCOMM 2014把必须全部完成的一组流作为 coflow，联合考虑发送与接收资源。这为本项目同时检查 SSD 和 NPU 接收链路提供了相近的分析视角，但其网络模型和优化目标并不等同本模拟器。本文的具体性能结论来自仓库结果审计与实际仿真，不借用论文的性能数字或算法保证。

原始 data 是画像参数表，不是生产到达统计；1K 外推、配额、固定角色、同序循环和到达时钟均属构造。策略计算与跨客户端协调成本未注入模拟时间，因此这些结果证明本模型中的机制和收益范围，不能直接证明硬件部署收益。

复查入口

- notes/existing_audit.md：原实验完整审计、代码位置、同画像分解。
- notes/reaggregate_existing.py：重算 44 个既有 case 与源结果哈希。
- notes/lowutil_audit.md：旧坏例、已有相位探针与 10 个小型筛选。
- docs/experiment_plan.md：正式实验前的口径与限制。
- 根目录 run_baseline_npu32_stress.py：冻结输入构造和单 case 运行。
- 根目录 sweep_baseline_npu32_stress.py：可审查 JSON 计划、进程隔离、45 秒状态与失败保留。
- screen/inputs、screen/runs：第一轮完整输入和原始仿真结果。
- grid：受害者数量与大请求计算长度格点，包含过载点。
- notes/all_strategy_table.md：screen/formal/holdout 全部 106 作业，分别保留 fixed/pipeline、窗口负例与完整批次取舍。
- notes/mixed_profile_audit：多短流主实验 36 作业，冻结输入、逐画像、逐卡及两窗口完整表。
- notes/mixed_strict_ss_audit、notes/mixed_policy_audit：真正同 SS 类别复核及 S1-S3 混合流补充。
- notes/mixed_six_ssu_review.md：六盘四作业及同人口八盘配对。
- analysis/matched_audit.json、analysis/failures.json：最终事件重聚合与明确失败记录。
- completion.json、final_verification.json：实际完成时间、结果计数范围、初始源码与冻结输入核验。
- final_source_and_inputs.tar.gz：源码、data、构造/分析脚本、冻结输入和可执行计划；仿真原始结果仍在各阶段 runs/。

原始结果审计与新作业分别统计；formal 的本机 offload 是原计划的子集，安装到原作业目录后只计一次。远端一次 SSH 传输中断未中止或重启后台模拟，原状态、传输事件与最终校验均留档。采集状态中的历史 queued/running 单独保留，不计为模拟失败；未完成结果也不填零。最终完整性以已校验的结果、实际返回码和冻结计划为准。